

Radar Posture Classification: A Comparative Study of State-of-the-Art Architectures

Eloan Tourtelier
CentraleSupélec
eloan.tourtelier@student-cs.fr

Othmane Ouali
CentraleSupélec
othmane.ouali@student-cs.fr

Lucas Cheylan
CentraleSupélec
lucas.cheylan@student-cs.fr

Meriem Mellouki
CentraleSupélec
meriem.mellouki@student-cs.fr

Zaineb Tarhda
CentraleSupélec
zaineb.tarhda@student-cs.fr

CONTENTS

1 Datasets used	1
2 PointNet	3
3 Sketching	5
4 Pointwise Attention Pooling	7
5 Skeleton Classifier with DGCNN	9
6 Classical Baselines	11
7 Moments	12
8 Results	13
9 Conclusion	13
A Appendix	14

ABSTRACT

In this work, we study and benchmark different deep learning architectures for human activity recognition (HAR) tasks (e.g., running, jumping, sitting, etc.) using radar data. Point cloud data offer strong potential due to their privacy-preserving and environment-agnostic properties (lighting conditions, etc.). However, point cloud-based HAR faces challenges such as data sparsity, high computational cost, and a lack of large annotated datasets.

The methods we study in this paper include PointNet [8], a skeleton predictor based on DGCNN [13, 14], methods using moment characterization, sketching, and simpler machine learning classifiers used as baselines. We benchmarked our models on four datasets, all recorded with mmWave radars and containing 2 to 10 classes. Our results highlight which architectural choices are most effective for sparse radar point data and provide practical guidance for future radar-based human state recognition. More importantly, they show that benchmark accuracy alone is not enough: for radar HAR, the data split, subject diversity, and temporal windowing protocol can matter as much as the architecture itself.

1 DATASETS USED

We studied four radar point datasets: RadHAR, DGUHA, MARS, and a dataset we recorded ourselves, which we named RaDataTouille. They all contain different classes and different features, but all of them have a time dimension, which some architectures use and some do not. Each radar snapshot is called a frame, and each frame contains 4 to 40 radar points, which requires a strategy for producing fixed-size inputs. A group of frames is called a window, and windows contain a fixed number of frames. We aim to keep our windows around 2 seconds long, which is a realistic time for our system to predict the activity the user is performing in an industrial scenario.

Source code. The RadHAR, RaDataTouille, and MARS datasets are available in this repository: `datasets/`. The DGUHA dataset can be downloaded from the link in the repository README.

1.1 RadHAR

The RadHAR dataset, introduced by Singh *et al.* [11] at the 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems (mmNets, 2019), is one of the most widely used public benchmarks for human activity recognition (HAR) from millimeter-wave (mmWave) radar point clouds. It is also referred to in the literature as the *MMActivity* dataset.

Dataset	Radar used	Classes	Total frames	Features
RadHAR / MMAActivity	TI IWR1443BOOST	5	167,632	x, y, z , range, velocity, bearing, intensity
RaDataTouille	TI xWR68xx AOP	2	27,411	x, y, z , SNR, velocity, noise
DGUHA	TI IWR1443BOOST mmWave radar	7	307,200	x, y, z
MARS	TI IWR1443BOOST mmWave radar	10	40,083	x, y, z , Doppler velocity, intensity

Acquisition setup. The data were collected with a Texas Instruments IWR1443BOOST mmWave FMCW radar, a low-cost, commercial off-the-shelf sensor operating in the 76–81 GHz band. The radar was mounted on a tripod at a height of 1.3 m, facing the subject. The sampling rate is approximately 30 Hz, i.e. one frame every ≈ 33 ms.

Activities and subjects. Two participants performed five activities in front of the radar: boxing, jumping, jumping jacks, squats, and walking. The dataset is shipped with a fixed train/test split, provided as separate `train/` and `test/` folders in the official repository. We use this split as-is throughout the paper, so that all our experiments remain comparable with prior work using RadHAR. The limited number of subjects (only two) is a critical caveat, however. Even with a clean train/test separation, the model can exploit subject-specific cues—gait rhythm, body size, characteristic arm swing. Near-ceiling accuracy on RadHAR should therefore be read as performance on a *closed-world* two-subject task, not as evidence of generalizable activity recognition.

For every detected point, the following features are provided:

Feature	Description
x, y, z	Cartesian coordinates of the point (m)
r (range)	Radial distance from the radar (m)
v (velocity)	Doppler radial velocity (m/s)
θ (bearing)	Azimuth angle (rad)
I (intensity)	Reflected signal intensity

These features are not independent: Cartesian coordinates (x, y, z) , range r and bearing θ are linked by the spherical-to-Cartesian relation, so the raw representation is *redundant*. Which subset is fed to a model is therefore a non-trivial design choice. In practice we found the five-dimensional representation (x, y, z, v, I) sufficient, discarding the redundant polar coordinates. The different feature combinations we explored are detailed in Section 3.1.

Effective dataset size. After windowing, the 167,632 frames yield approximately 2,793 labeled samples of 60 frames each. This is the quantity that actually limits classifier capacity: a model trained on this dataset sees roughly 560 samples per class on average, which is modest by deep-learning standards and makes per-subject variability a real concern given that only two subjects contributed data.

Split	Files	Frames
Train	142	128,947
Test	58	38,685
Total	200	167,632

Temporal windowing. Following the original RadHAR protocol, activities are classified over a sliding time window of 2 seconds, which at 30 fps corresponds to a sequence of 60 consecutive frames. Each such window constitutes one labeled sample for the classifier. The resulting object is therefore a *sequence of sets*: 60 unordered, variable-size point sets, collectively carrying a single activity label.

1.2 RaDataTouille

RaDataTouille is the dataset we recorded ourselves with a Texas Instruments xWR68xx AOP radar operating at 60 GHz. The AOP (Antenna-on-Package) form factor integrates the antenna array directly on the chip, making the sensor significantly more compact than the tripod-mounted IWR1443BOOST used for RadHAR and MARS. This compactness is directly relevant to the project’s target application: a sensor that must be embedded in furniture or worn by a user cannot use a large benchtop board.

The dataset focuses on a binary posture-recognition problem with two labels: `sit` and `stand up`. Although this is a simpler classification problem than the five- or ten-class benchmarks, we recorded the postures in quite difficult environments, with varied radar angles and spatial configurations (using obstacles, walls, and different distances between the radar and the subject, from 50 cm to 3 meters). We recorded three subjects: Eloan, Othmane, and Lucas.

Each detected radar point is represented by six features:

$$(x, y, z, \text{snr}, \text{velocity}, \text{noise}).$$

The split contains 13 training records and 6 test records, totaling 19 recordings, 20,031 training frames and 7,380 test frames (27,411 frames overall). At roughly $15\times$ smaller than RadHAR by frame count, it is a meaningful stress test of whether the studied architectures generalize under limited supervision. The recordings are handled at 10 fps in our tooling, so a 2-second window corresponds to 20 frames.

Split	Records	Frames
Train	13	20,031
Test	6	7,380
Total	19	27,411

1.3 DGUHA

DGUHA, the Dongguk University Human Activity dataset, was introduced by Lee and Kim in the MTGEA paper [7]. The dataset and code are available from the authors’ MTGEA repository.¹ It contains synchronized radar point clouds and Kinect recordings from 19 subjects in their twenties, with an average age of 23, and seven activity labels: running, jumping, sitting down and standing up, both upper-limb extension, falling forward, right-limb extension and left-limb extension.

Each radar point used in this repository carries three features: Cartesian coordinates (x, y, z) .

Each clip consists of 400 frames, and each frame is fixed at 25 points. Unlike RadHAR’s sliding-window protocol, each clip is a complete, semantically bounded action: the 400-frame window was chosen to span one full execution of the movement. Since each activity recording lasts approximately 20 seconds, DGUHA is about 20 fps, so a 2-second window corresponds to about 40 frames. The train/test split contains 607 training clips and 161 test clips (768 total, 307,200 frames). The sensor is a Texas Instruments IWR1443B00ST mmWave radar—the same board used for RadHAR and MARS—paired with an Azure Kinect for ground-truth annotations.

Split	Clips	Frames
Train	607	242,800
Test	161	64,400
Total	768	307,200

1.4 MARS

MARS, for *mmWave-based Assistive Rehabilitation System for Smart Healthcare*, is a rehabilitation-oriented dataset containing synchronized mmWave radar data and Kinect skeleton annotations [2]. It was collected with a Texas Instruments IWR1443B00ST mmWave radar and targets ten rehabilitation movements: left upper-limb extension, right upper-limb extension, both upper-limb extension, left front lunge, right front lunge, squat, left side lunge, right side lunge, left-limb extension and right-limb extension.

Each radar point carries five features:

$$(x, y, z, \text{Doppler velocity}, \text{intensity}),$$

where x is the lateral axis, y is the depth axis and z is the vertical axis.

The repository split follows a *subject-based* protocol: recordings from different participants are assigned exclusively to train or test, so the evaluation measures genuine inter-subject generalization rather than interpolation within a single subject’s sessions. This is a stricter and more realistic split than a random frame or window partition. The split contains 29 training recordings and 10 test recordings, corresponding to 28,221 training frames and 11,801 test frames. The radar frame duration is 100 ms (10 fps), so a 2-second window corresponds to 20 frames.

¹<https://github.com/AIC-DGU/MTGEA>

Note on data leakage in published results. The original work associated with MARS reported results using a *frame-level* random split rather than a subject-based one. Because consecutive frames from the same recording are highly correlated — a window starting at frame t and one at frame $t + 1$ share 59 of their 60 frames — randomly partitioning at the frame level places nearly identical windows in both training and test sets. The model effectively memorizes specific movement executions rather than learning motion patterns that generalize across subjects. Results obtained under this split are substantially inflated and are not reproducible under a subject-isolated evaluation. All our own MARS experiments use the subject-based split described above.

Split	Recordings	Frames
Train	29	28,221
Test	10	11,801
Total	39	40,083

2 POINTNET

Student contribution: Eloan Tourtelier

PointNet [8] is a fundamental model in point-cloud deep learning. It was built to process an unordered set of 3D points with no pixel grid and no voxel ordering. In this project, we use two variants. V1 is the original vanilla PointNet classifier, which we validated on the ModelNet40 3D object benchmark from the original PointNet paper [8]. V2 adds explicit temporal processing (over the sequence of frames) using an attention mechanism, enabling each point from each frame to “talk” to itself across the time window.

Source code. The PointNet implementations and training notebooks are available in the project repository: `src/pointnetV1/` and `src/pointnetV2/`.

2.1 Vanilla PointNet

PointNet processes each radar frame as an unordered point set. The point cloud is first processed by a T-Net, which multiplies the distribution by a matrix to rotate/reflect the data. Each point is then processed independently by a stack of shared MLPs that increase the dimensionality to 1024. This process is repeated twice in the original paper, but this can be increased. A global max-pooling operation then collapses along the point dimension into a single 1024-dimensional descriptor. Because max-pooling is invariant to permutation, it is invariant to the order in which points are presented. This permutation invariance is critical for every model processing radar points, because detectors return points in arbitrary order. Finally, a three-layer MLP head maps the max-pooled output to class logits. The detailed V1 module layout and hyperparameters are given in Appendix Table 14.

About the two transformation networks (T-Nets): The input T-Net predicts a $D \times D$ (the feature dimension) matrix that is applied to the raw point coordinates, canonicalizing the point cloud orientation before any learned embedding. Both matrices are regularized in the loss with an orthogonality penalty

($\lambda = 0.001$) to keep the learned transformations close to a rotation and/or a reflection, keeping lengths, angles, and volumes unchanged; without this constraint, the high-dimensional feature T-Net would overfit to arbitrary linear maps, while we only want the T-Net to stick to reorienting the distribution. An orthogonal matrix A verifies

$$AA^\top = I,$$

and is implemented in the loss as a deviation from the identity matrix, $\|I - AA^\top\|_F^2$.

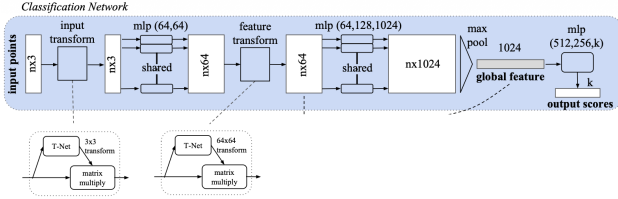


Figure 1: PointNet architecture. The T-Net modules canonicalize point-cloud orientation, while global max pooling produces an order-invariant descriptor.

2.2 Attention PointNet

Within a single frame, points are an unordered set and must be treated symmetrically; across frames, there is a meaningful ordering, which carries information. V2 addresses this by stacking a temporal attention mechanism inside the PointNet encoder, right after the T-Net rotation.

For the attention mechanism, we use multi-head self-attention with 8 heads or fewer (depending on whether the dimension is large enough for 8 heads). Each frame can attend to all others and weight the time steps most informative for the activity decision. Positional information is injected via *Rotary Position Encoding* (RoPE), which encodes relative temporal distances rather than absolute positions between frames. We add an attention layer after each T-Net, and therefore have two blocks: T-Net $\rightarrow$ time attention $\rightarrow$ MLP. The time dimension is mean pooled at the very end, after max pooling (along the N dimension), and right before the final classification head.

The addition of the time attention mechanism is not always beneficial. The longer the time window is (the more frames it contains), the better the accuracy tends to be. However, for practical reasons, we keep our windows around 2 seconds long, which is the time one would expect the radar to classify an input in a commercial scenario. The detailed V2 module layout and hyperparameters are given in Appendix Table 15.

2.3 Data Processing

Normalization. Per-feature standardization is applied first: each feature dimension is shifted to zero mean and rescaled to unit variance using statistics computed on the *training data only*. The same mean and standard deviation are stored and reapplied at test time, so no test-distribution information leaks into the preprocessing pipeline. Outliers are then clipped to be bounded by four times the standard deviation.

Rolling windows. Each recording is segmented into overlapping windows using a sliding stride smaller than the window length. The stride is set to a fraction of the window size (1/2 to 1/3), so consecutive windows share some of their frames but start at different times in the cycle. This multiplies the number of labeled samples, but also extracts more information from the data, as the classifier becomes more agnostic to the beginning of the recording.

Augmentation. All four datasets share two structural challenges: small subject counts and limited recording diversity. Data augmentation is therefore essential to prevent models from overfitting to subject-specific radar signatures. We apply a common augmentation pipeline during training only. The pipeline operates on raw point clouds before any architecture-specific preprocessing, and contains scaling, offsetting, and jittering.

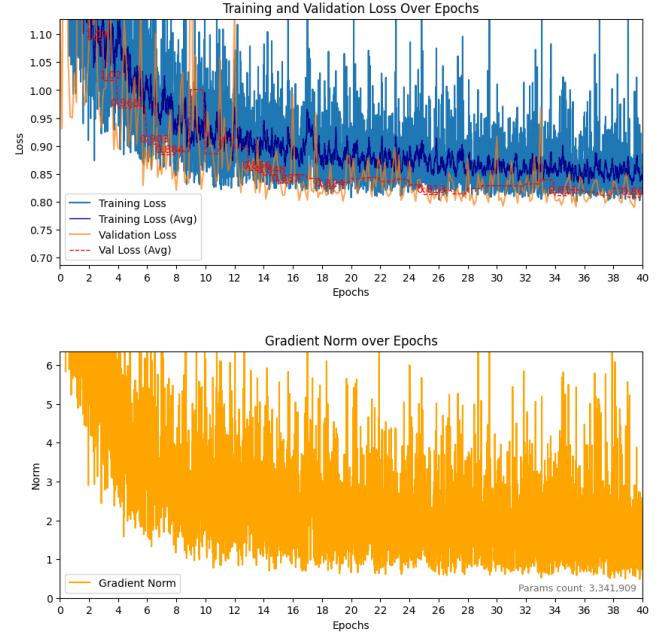


Figure 2: When adding augmentation to the training process, we observe that the training losses are above the validation losses at first, until the model overfits and the training losses fall below the validation losses.

Scale. At each training step, all spatial coordinates (x, y, z) of a window are multiplied by a scalar drawn uniformly from $[0.85, 1.15]$. This simulates subjects of different heights and body sizes as well as small calibration offsets in radar mounting distance, reducing the tendency of models to exploit absolute distance as a class cue.

Offset. A random translation vector with each component drawn from $\mathcal{U}(-0.1 \text{ m}, 0.1 \text{ m})$ is added to every point in the window. This encourages position invariance within the sensor field of view and compensates for the fixed radar placements across datasets.

Point jitter. Independent Gaussian noise $\varepsilon \sim \mathcal{N}(0, 0.005^2)$ is added to each coordinate of each point. The amplitude is kept small relative to the unit variance of the normalized data so that local geometry is preserved while the model becomes robust to sensor noise and single-frame detection instability.

2.4 Results

Here are the final accuracies and F1 scores obtained on each of the four datasets. We used five different initializations with five seeds to make the results reproducible. We observe that the initialization has a strong impact on the scores. For all classification results, accuracy is the percentage of test windows that are classified correctly.

Macro F1 gives equal weight to each class by averaging class-wise F1:

$$F1 = \frac{1}{C} \sum_{c=1}^C \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

F1 is especially useful when datasets are imbalanced, which is not the case for our datasets. We still compute it, as it provides information when one class is less well classified than the others.

Seed	V1		V2	
	Acc.	F1	Acc.	F1
1234	0.9561	0.9572	0.9624	0.9634
1235	0.9592	0.9605	0.9561	0.9567
1236	0.9624	0.9637	0.9624	0.9632
1237	0.9451	0.9470	0.9765	0.9770
1238	0.9514	0.9526	0.9671	0.9675
Mean	0.9548	0.9562	0.9649	0.9656

Table 1: PointNet V1 vs. V2 on RadHAR

Seed	V1		V2	
	Acc.	F1	Acc.	F1
1234	0.9255	0.9263	0.9441	0.9449
1235	0.9193	0.9199	0.9503	0.9514
1236	0.8944	0.8981	0.9006	0.8960
1237	0.9379	0.9363	0.9441	0.9448
1238	0.9193	0.9217	0.9006	0.9001
Mean	0.9193	0.9205	0.9279	0.9274

Table 2: PointNet V1 vs. V2 on DGUHA

Seed	V1		V2	
	Acc.	F1	Acc.	F1
1234	0.8274	0.8222	0.8685	0.8674
1235	0.8274	0.8222	0.8430	0.8412
1236	0.8402	0.8359	0.6959	0.6902
1237	0.8062	0.7992	0.7893	0.7890
1238	0.8274	0.8222	0.8119	0.8113
Mean	0.8257	0.8203	0.8017	0.7998

Table 3: PointNet V1 vs. V2 on RaDataTouille

Seed	V1		V2	
	Acc.	F1	Acc.	F1
1234	0.8917	0.7891	0.8689	0.7718
1235	0.8958	0.7928	0.7951	0.7103
1236	0.8792	0.7777	0.9016	0.8000
1237	0.8833	0.7837	0.8115	0.7224
1238	0.9000	0.7964	0.8525	0.7573
Mean	0.8900	0.7900	0.8455	0.7524

Table 4: PointNet V1 vs. V2 on MARS

RadHAR. Both PointNet variants perform strongly on RadHAR. V2 averages 96.5% accuracy and 0.9656 macro F1 across five seeds, slightly above V1, which averages 95.5% accuracy and 0.9562 macro F1. The difference is small relative to the near-ceiling scores, but V2 is more stable on the best run, reaching 97.7% accuracy and 0.9770 macro F1 at seed 1237.

DGUHA. V2 outperforms V1 by +0.86 pp in accuracy and +0.69 pp in macro F1 on average. The gain is consistent across seeds 1234, 1235, and 1237; seed 1236 is a weak run for both models but V2 still matches V1. The long 400-frame windows in DGUHA give the temporal attention enough sequence to exploit motion dynamics across the 7 activity classes.

RaDataTouille. The picture is reversed: V1 averages 82.6% across all five seeds, while V2 averages 80.2% (or 82.8% excluding the exploding-gradient run at seed 1236). The binary sit/stand task is small and the recordings are short (20-frame windows), leaving little temporal structure for the attention mechanism to leverage. In this low-data regime, the extra parameters of V2 appear to hurt rather than help, consistent with the MARS observation in Section 8.

MARS. V1 is also stronger on MARS, averaging 89.0% accuracy and 0.7900 macro F1, compared with 84.6% accuracy and 0.7524 macro F1 for V2. The validation accuracies are close to perfect for both variants, while test macro F1 remains lower because both models fail to recognize the left and right limb extension classes in the subject-isolated test set. This gap suggests that MARS is not only an optimization problem, but also a generalization problem: the models fit the training subjects well, yet struggle when the same rehabilitation gestures are performed by an unseen subject.

3 SKETCHING

Student contribution: Othmane Ouali

3.1 Preprocessing: Random Fourier Feature sketching

Why preprocessing is needed. A raw radar frame is an *unordered set of points of variable cardinality*. No standard learning architecture — be it an MLP, a CNN, a Transformer or a gradient-boosted tree — can consume such an object directly: they all expect inputs of fixed shape and well-defined structure. A preprocessing step is therefore unavoidable. The

original RadHAR baseline addresses this issue through *voxelization*: each 2-second window is discretized into a tensor of shape $60 \times 10 \times 32 \times 32 = 614400$ scalar features [11]. This representation is extremely high-dimensional and overwhelmingly sparse, since a typical frame contains only a few tens of points scattered in a grid of 10,240 voxels. The downstream model thus has to invest most of its capacity in processing empty space.

We propose instead to compress each frame into a compact, fixed-size vector via a *random Fourier feature map*, which preserves the information carried by the points while keeping the dimensionality manageable.

Random Fourier feature sketching. For a given choice of d per-point features (see paragraph *Feature selection* below), a frame at time t is a point set $\mathcal{P}_t = \{p_1, \dots, p_{N_t}\} \subset \mathbb{R}^d$, where N_t varies from frame to frame. We map this set to a fixed-size sketch $s_t \in \mathbb{R}^m$ in two steps.

Step 1 — Per-point random projection. We draw a single random matrix $W \in \mathbb{R}^{m \times d}$ with i.i.d. entries

$$W_{ij} \sim \mathcal{N}(0, \sigma^2),$$

once and for all at the beginning of training, and freeze it. Each point p_i is mapped to

$$\varphi(p_i) = \sin(W p_i) \in \mathbb{R}^m.$$

This is a Fourier-type random feature map (with bias $b = 0$ and using the sine component only), inspired by the random Fourier feature construction of Rahimi and Recht [9]. With $b = 0$ and a single trigonometric component, the inner product $\langle \varphi(p_i), \varphi(p_j) \rangle$ does not strictly approximate a Gaussian kernel; we use it nonetheless as a randomized, non-linear feature map, which proved sufficient for our task.

Step 2 — Aggregation across the frame. Since $\mathcal{P}_t$ is a set, the aggregation across its points must be permutation-invariant. We consider two variants:

- **Uniform sketch:** $s_t = \frac{1}{N_t} \sum_{i=1}^{N_t} \sin(W p_i)$.
- **Intensity-weighted sketch:** $s_t = \frac{1}{\sum_i I_i} \sum_{i=1}^{N_t} I_i \sin(W p_i)$,

where I_i denotes the reflected intensity of point i . The intensity-weighted variant gives more importance to high-confidence reflections, which in our experiments proved beneficial.

The whole window is then represented by a tensor of shape $60 \times m$, obtained by stacking the 60 per-frame sketches.

Feature standardization. Per-point features live on widely different scales: range is in metres ($r \in [0, 10]$ approximately), bearing in radians ($\theta \in [-\pi, \pi]$), velocity in m/s, intensity on an arbitrary scale. Because the random projection $W p_i$ mixes all features linearly, this scale heterogeneity would make the choice of σ ill-defined. We therefore standardize every feature to zero mean and unit variance *before* applying the sketching

step. The per-feature mean μ and standard deviation s are computed *on the training set only*, and the same statistics are reused at test time, so that no information leaks from the test set into the preprocessing pipeline.

Feature selection. The seven raw features per point $(x, y, z, r, v, \theta, I)$ are redundant: (x, y, z) , r and θ are linked by the spherical-to-Cartesian relation. Determining which subset is most informative is therefore an empirical question. We compared four configurations:

Configuration	Per-point features	Aggregation
All features	$(x, y, z, r, v, \theta, I)$	uniform
Cartesian + I	(x, y, z, I)	uniform
Polar + I	(θ, r, I)	uniform
Polar + I (weighted)	(θ, r, I)	I -weighted

The polar configuration (θ, r, I) consistently outperformed the Cartesian variants, which motivated the introduction of the intensity-weighted sketch as a refinement of that configuration. A detailed comparison is reported in the ablation study (Section 3.4).

Sliding-window construction. Each raw recording is segmented into 2-second windows (60 consecutive frames) using a sliding window with a stride of 10 frames. This yields a controlled overlap (50 frames, $\approx 83\%$) between consecutive windows: small enough to keep them distinct from the classifier’s point of view, large enough to make full use of the available recordings. The procedure produces a total of 12 150 training windows and 3 565 test windows, in line with the official RadHAR train/test split.

Hyperparameter choices. The preprocessing pipeline introduces several hyperparameters whose impact is non-trivial: the random-projection scale σ , the sketch dimension m , the sliding-window stride, the per-point feature subset, and the aggregation scheme (uniform vs. intensity-weighted). All of them were studied empirically; the corresponding ablation study is reported in Section 3.4. Unless stated otherwise, the rest of the paper uses $\sigma = 1$, $m = 256$, polar features (θ, r, I) , intensity-weighted aggregation, and a stride of 10 frames.

Summary. The full preprocessing pipeline can be summarized as

$$\begin{aligned} \text{recording} &\rightarrow \text{frames} \rightarrow 2\text{-s windows} \\ &\rightarrow \text{sketched windows} \rightarrow \text{model input.} \end{aligned}$$

The data finally fed to all models compared in this paper is therefore a tensor of shape $(12\,150, 60, 256)$ for training and $(3\,565, 60, 256)$ for testing.

3.2 Transformer encoder

Overview. The preprocessing stage turns each 2-second window into a sequence of 60 sketches $s_1, \dots, s_{60} \in \mathbb{R}^m$, one per

frame. We treat this sequence as a sequence of 60 tokens and process it with a Transformer encoder, which models temporal dependencies between frames through self-attention. We deliberately do not introduce an input projection layer: the sketches are fed directly to the encoder, so that the model dimension equals the sketch dimension, $d_{\text{model}} = m$. As a side effect, varying m in the ablation study (Section 3.4) also varies the size of the Transformer.

Positional encoding. Self-attention is permutation-invariant: it would treat the 60 frames as an unordered set and discard the temporal order that distinguishes activities such as walking from boxing. To restore the temporal order, we add a fixed sinusoidal positional encoding [12] to each sketch before the first encoder layer:

$$\tilde{s}_t = s_t + \text{PE}(t), \quad t = 1, \dots, 60.$$

For each position t and each dimension $i = 0, \dots, m-1$:

$$\text{PE}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/m}}\right), \quad \text{PE}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/m}}\right).$$

This encoding is computed once and kept fixed throughout training.

Encoder block. Each encoder layer follows the standard Transformer [12] design: a multi-head self-attention sub-layer followed by a position-wise feed-forward network, each wrapped in a residual connection and layer normalization (post-norm). The self-attention uses $h = 4$ heads; the feed-forward sublayer has a single hidden layer of width $2m$ with a GELU activation. Dropout with rate $p = 0.4$ is applied inside each sublayer. We stack $L = 3$ such layers.

Classification head. After the encoder layers, we obtain a sequence of contextualized frame representations $h_1, \dots, h_{60}$. We aggregate them by mean pooling over the temporal axis,

$$\bar{h} = \frac{1}{60} \sum_{t=1}^{60} h_t \in \mathbb{R}^m,$$

and feed the result to a linear layer that outputs the activity logits. We use mean pooling rather than a dedicated [CLS] token because it gave comparable results without introducing an extra learned token.

Hyperparameters. The architecture is governed by four hyperparameters: the model dimension $m = 256$, the number of encoder layers $L = 3$, the number of attention heads $h = 4$, and the dropout rate $p = 0.4$.

The full architecture is summarized in Figure 9 (Appendix A.1).

3.3 Training protocol

Loss and optimization. We minimize the cross-entropy loss between predicted logits and ground-truth activity labels. Optimization is performed with Adam at a fixed learning rate of

10^{-4} , with no weight decay and no learning-rate schedule. We use a mini-batch size of 32 and train for 30 epochs, without early stopping.

Weight initialization. All trainable weight matrices in the Transformer encoder and final classification head are initialized with Kaiming initialization; biases are initialized to zero. The random-projection matrix W of the sketching step is not trained: it is drawn once from $\mathcal{N}(0, \sigma^2)$ at the start of training and kept fixed.

Evaluation protocol. We use the official RadHAR train/test split (12 150 training windows and 3 565 test windows) without carving out a separate validation set. The final test accuracy reported in the main comparison corresponds to the model state at the last training epoch.

Training dynamics. The training loss decreases smoothly toward zero, while the test loss decreases overall but exhibits large epoch-to-epoch variance. This pattern is not a typical sign of overfitting, where the test loss would increase monotonically; we attribute it primarily to a distributional gap between the training and test splits of RadHAR.

Missing local file: loss.png

Figure 3: Training and test cross-entropy loss across the 30 epochs. The training loss converges smoothly toward zero, while the test loss exhibits large epoch-to-epoch variance, attributable to the distributional gap between the train and test splits of RadHAR.

3.4 Ablation studies

4 POINTWISE ATTENTION POOLING

Student contribution: Lucas Cheylan

This section presents the raw-point attention model studied as a direct classifier for radar activity recognition. It is designed for sparse radar sequences, where each recording is a temporal window of frames and each frame contains an unordered set of radar detections.

4.1 Input representation

For every dataset, the ingestion scripts convert the raw files into the same working format: a sequence of radar frames. Each frame contains points described by their position, velocity and radar response,

$$p_i = (x_i, y_i, z_i, v_i, I_i).$$

The number of detections varies from one frame to another. To obtain fixed-size tensors, we keep at most 32 points per frame and pad shorter frames with empty points. A binary mask is carried through the model so that padded entries do not contribute to the attention weights or to the pooled descriptors.

Training samples are built from windows of 60 consecutive frames with stride 10. The resulting input tensor has shape

$$X \in \mathbb{R}^{B \times T \times P \times F},$$

where $T = 60$ is the number of frames, $P = 32$ is the maximum number of points per frame, and $F = 5$ is the number of point features.

4.2 Model mechanism

The model is designed to process radar point clouds without imposing a fixed order on the detections. In each frame, the radar returns a set of points, and this set can change in size from one frame to another. The first goal of the model is therefore to transform a variable and unordered set of points into one stable frame descriptor. The second goal is to use the sequence of frame descriptors to recognize the activity over time.

For each frame t , every point $p_{t,i}$ is first passed through the same pointwise multilayer perceptron (MLP):

$$e_{t,i} = \phi(p_{t,i}),$$

where $e_{t,i}$ is the learned embedding of point i in frame t . The MLP is shared across all points and all frames. This is important because the order of the points in the input tensor is artificial: point 1 is not more important than point 2 only because it appears earlier in the array. By applying the same function ϕ to every point, the model learns a representation of each radar detection independently from its position in the list.

After this pointwise embedding step, the model applies an intra-frame attention pooling mechanism. The role of this layer is to decide which points are useful inside one frame. Each embedded point receives an attention score:

$$q_{t,i} = w_p^\top \tanh(W_p e_{t,i} + b_p),$$

where W_p , b_p and w_p are learned parameters. These scores are then normalized with a softmax over the points of the same frame:

$$\beta_{t,i} = \frac{\exp(q_{t,i})}{\sum_{j=1}^P \exp(q_{t,j})}.$$

In practice, the binary mask is applied before the softmax. Padded points are given a very small score, so their final attention weight is close to zero. This prevents empty points from influencing the representation.

The frame descriptor is then computed as a weighted sum of the point embeddings:

$$z_t = \sum_{i=1}^P \beta_{t,i} e_{t,i}.$$

This vector z_t summarizes the useful radar information in frame t . Points that look like noise, missing detections, or weak background reflections can receive low weights. Points that correspond to the body, limbs, or visible motion can receive higher weights. The model does not need hand-crafted rules to select these points; the selection is learned directly from the classification loss.

This first attention stage also gives the model a form of permutation invariance. If the valid points of one frame are reordered, the same pointwise MLP is still applied to each point, and the final pooling operation is a sum. Therefore, the

frame descriptor depends on the content of the point cloud, not on the arbitrary order used to store it.

Once each frame has been converted into a descriptor z_t , the full input window becomes a temporal sequence:

$$(z_1, z_2, \dots, z_T).$$

This sequence is passed to a bidirectional GRU. The GRU models how the radar scene evolves during the activity. The forward direction captures information from the beginning to the end of the window, while the backward direction uses the same window in the opposite direction. The hidden state used at time t is therefore

$$h_t = [\vec{h}_t; \overleftarrow{h}_t],$$

where the two directional states are concatenated.

The GRU output still contains one vector per frame. However, not all frames are equally useful for recognizing the activity. Some frames may correspond to a transition, a pause, or a moment where the radar signal is weak. For this reason, a second attention pooling layer is applied over time. It computes a temporal score for each GRU state:

$$r_t = w_r^\top \tanh(W_r h_t + b_r),$$

and normalizes these scores across the T frames:

$$\alpha_t = \frac{\exp(r_t)}{\sum_{k=1}^T \exp(r_k)}.$$

The complete sequence is then represented by a single descriptor:

$$s = \sum_{t=1}^T \alpha_t h_t,$$

This descriptor gives more importance to the frames that are most useful for the final prediction.

The final classifier is a small feed-forward head applied to s . It produces one logit per activity class:

$$\hat{y} = W_c s + b_c,$$

and the class probabilities are obtained with a softmax. During training, the model is optimized with the cross-entropy loss between the predicted class distribution and the ground-truth activity label.

The complete mechanism can be understood as a two-level learned pooling model. At the first level, the model learns which radar points matter inside each frame. At the second level, it learns which frames matter inside the temporal window. This structure is well suited to radar activity recognition because the signal is sparse, noisy, and time-dependent. It keeps the raw detections instead of replacing them with hand-crafted statistics, but it remains much lighter than a full point-cloud backbone. The attention weights also make the architecture more interpretable: the model explicitly shows where it pools information, first over points and then over time.

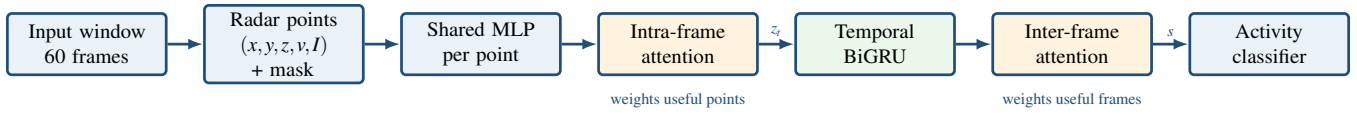


Figure 4: Architecture of the pointwise attention pooling model. The model first learns which radar points to keep within each frame, then learns which frames matter most for the final activity prediction.

4.3 Experimental protocol

The same reference pipeline was run on four cleaned datasets using their native train/test splits. Although the raw formats differ across datasets, the preprocessing step maps all of them to the same frame-based representation with points (x, y, z, v, I) . The evaluation protocol is then shared: windows of 60 frames, stride 10, up to 32 points per frame, and 12 training epochs.

4.4 Discussion

Dataset	Classes	Test windows	Test accuracy	Macro F1
RaDataTouille	2	2,319	76.46%	0.7643
MARS	10	3,877	82.51%	0.8276
RadHAR	5	3,565	95.20%	0.9523
DGUHA	7	8,372	91.61%	0.9166

Table 5: Results of the pointwise attention pooling model on four radar datasets.

The results in Table 5 show that this mechanism is strong on RadHAR and DGUHA, while MARS and RaDataTouille remain more difficult. RadHAR is the most favorable setting for the model, with a test accuracy of 95.20%. DGUHA also gives a high score, although it was run on GPU with a larger batch size to keep training time practical. MARS is harder because it contains ten rehabilitation movements with closer visual and kinematic patterns. RaDataTouille is smaller and targets a sit-versus-stand setting, so the model has less data from which to learn robust temporal cues.

This baseline is useful because it keeps the raw radar points while avoiding a large point-cloud backbone. Compared with a moment-based model, it can use the relative importance of individual detections. Compared with heavier architectures, it is easier to train and gives a clear interpretation of where attention is applied: first over points, then over time.

5 SKELETON CLASSIFIER WITH DGCNN

Student contribution: Eloan Tourtelier

The SK-DGCNN approach follows a sequential two-stage strategy adapted from the SK-DGCNN method [14]: instead of classifying sparse radar points directly, it first maps each radar frame to a structured human skeleton and then classifies the resulting skeleton. This approach has been described as successful in many cases, and it gives the model more information in principle, especially about skeleton feasibility: for example, a foot cannot be 1 meter away from the knee. It also enables the use of external datasets, counterbalancing data sparsity issues for the target dataset.

Source code. The skeleton pipeline implementation and training notebooks are available in the project repository: [src/SK-dgcnn/](https://github.com/eloan-tourtelier/sk-dgcnn).

5.1 Radar-to-skeleton stage

The first stage is a skeleton-aware DGCNN regressor trained on MARS [2]. DGCNN is a very strong model, built as a generalization of PointNet [8] by introducing spatial neighborhoods using graphs [13]. In MARS, radar frames are synchronized with Kinect skeleton annotations, which record the real skeleton positions. A dynamic k -nearest-neighbor graph is recomputed inside the network, and three EdgeConv blocks extract local geometric features. Efficient channel attention is added after each block so that the network can reweight the most informative channels before global pooling.

The output is a centered 19-joint skeleton using the lateral and vertical coordinates (x, z) , which the original paper reports as better than the 3D representation, giving 38 regression targets per frame. We center the skeleton around the SpineShoulder joint so that the regressor focuses on body pose rather than absolute position in the radar field of view. The pose regressor follows the SK-DGCNN paper’s shape-aware objective: a coordinate-wise MSE term keeps individual joints close to their Kinect target, while a KL-divergence term treats the predicted and target skeleton coordinates as spatial distributions and penalizes global shape mismatch. For one active axis $c \in \{x, z\}$ with predicted coordinates $\hat{s}_{i,c}$ and target coordinates $s_{i,c}$ over $J = 19$ joints,

$$\mathcal{L}_{\text{MSE},c} = \frac{1}{J} \sum_{i=1}^J (\hat{s}_{i,c} - s_{i,c})^2,$$

$$\mathcal{L}_{\text{KL},c} = \sum_{i=1}^J q_{i,c} \log \frac{q_{i,c}}{p_{i,c}},$$

$$p_c = \text{softmax}(\hat{s}_{:,c}),$$

$$q_c = \text{softmax}(s_{:,c}).$$

The mixed loss is

$$\mathcal{L}_c = \alpha_{\text{MSE}} \mathcal{L}_{\text{MSE},c} + \alpha_{\text{KL}} \mathcal{L}_{\text{KL},c},$$

with dynamic weights

$$\alpha_{\text{KL}} = 0.05 \left\lfloor \frac{\text{epoch}}{20} \right\rfloor, \quad \alpha_{\text{MSE}} = 1 - \alpha_{\text{KL}}.$$

In our two-axis setup, the total loss sums the active axes, $\mathcal{L}_{\text{total}} = \sum_{c \in \mathcal{A}} \mathcal{L}_c$, where $\mathcal{A}$ grows from z to (z, x) during sequential multitask training. The architecture and training

hyperparameters for this stage are summarized in Appendix Table 16. This regression stage is the hardest in the sequence and requires very compute-intensive training (200 epochs).

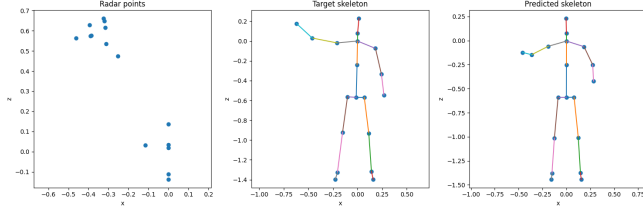


Figure 5: Example of a radar frame converted into a centered 19-joint skeleton representation.

5.2 Skeleton classification stage

Once the radar-to-skeleton checkpoint is trained, it can be used to transform radar recordings into skeleton sequences. Dataset windows are converted from raw radar point clouds into predicted 19-joint skeletons. The activity classifier then operates on this structured sequence rather than on the original point cloud. This separates the perception problem from the classification problem: the first network learns a radar-to-body representation, and the second network learns activity dynamics from body motion.

This design is expected to help when the activity is better explained by joint configuration than by the raw distribution of radar reflections. Its main drawback is error propagation: if the skeleton regressor fails on a frame, the classifier only sees the incorrect skeleton and cannot recover the discarded radar evidence.

5.3 Data Processing

Radar-to-skeleton preprocessing. For the pose-estimation stage, each MARS radar frame is matched with its synchronized Kinect annotation. Radar frames are cropped or padded to 25 points, using the five MARS radar features x, y, z , Doppler velocity and intensity. The Kinect target is reduced to the 19-joint contract used by SK-DGCNN and to the two coordinates (x, z) , corresponding to lateral and vertical motion. The Kinect lateral axis is inverted to align it with the radar coordinate frame, and the skeleton is centered on SpineShoulder.

Skeleton cache generation. After training the pose regressor on MARS, the checkpoint is applied to the other datasets to turn them into skeletons. Each radar frame is converted into a predicted 19-joint skeleton. The resulting representation keeps only body motion and discards the original radar points, producing sequences of centered lateral and vertical joint coordinates for classification.

Windowing and normalization. The classifier uses fixed-length skeleton windows from the cached sequences. Before classification, every skeleton is centered again around SpineShoulder. Training-set statistics are then used to standardize the lateral and vertical coordinates, and the same statis-

tics are reused at validation and test time. A constant confidence channel is concatenated to the two coordinate channels, so the ST-GCN++ input has three channels over time and over the 19-joint skeleton graph.

Augmentation. During classifier training only, small Gaussian jitter is added to the centered skeleton coordinates. The skeleton is re-centered after jittering so that the augmentation changes pose and local motion slightly without shifting the whole body in the coordinate frame.

5.4 Results

The skeleton pipeline is evaluated in two parts: first as a radar-to-skeleton regressor on MARS, then as a full skeleton-based activity classifier on the respective datasets. Central torso joints are reconstructed most accurately, while wrists and feet have larger errors because they move farther from the body center and are harder to infer from sparse radar reflections. For the skeleton regressor, mean absolute error is computed over the two predicted coordinates (z, x) :

$$\text{MAE} = \frac{1}{2NJ} \sum_{n=1}^N \sum_{i=1}^J \sum_{c \in \{z, x\}} \left| \hat{s}_{i,c}^{(n)} - s_{i,c}^{(n)} \right|,$$

where N is the number of frames and $J = 19$ is the number of joints.

Evaluation	Metric	Value
MARS pose regressor	Overall MAE	5.0456 cm
Best joint	SpineShoulder MAE	0.0645 cm
Worst joint	WristRight MAE	9.7054 cm

Table 6: SK-DGCNN radar-to-skeleton regression results on MARS.

Dataset	Validation acc.	Test acc.	Test macro F1
DGUHA	0.9013	0.8738	0.8741
RadHAR	0.9294	0.8877	0.8879

Table 7: Skeleton-based ST-GCN++ classification on predicted skeletons.

The downstream classifier performs well, but remains below the best direct PointNet variants on RadHAR and DGUHA. These results are far below those reported in the SK-DGCNN paper, where the reported accuracy and F1 are 99.5%. After requesting the authors’ source code and investigating possible data leakage, we found that this concern was justified. The original paper does not use the official test dataset; instead, it recreates a test set from the training dataset, making the task easier. Moreover, the paper first processes the data with rolling windows before splitting it into training and testing sets, causing windows in those sets to overlap substantially. Our results are therefore more meaningful, and they show that this approach, in its current form, underperforms the other models, even though it seemed very promising. For the global comparison, we count the full two-stage pipeline: the skeleton regressor plus the 1,081,093-parameter ST-GCN++ classifier.

This gives 3,014,547 parameters for the MARS-trained regressor pipeline and 3,016,979 parameters for the DGUHA-trained regressor pipeline.

6 CLASSICAL BASELINES

Student contribution: Meriem Mellouki and Zaineb Tarhda

In this section, we present the comparative performance of baseline models across the four radar datasets. The objective is to evaluate how far global statistical feature extraction can go as a lightweight alternative to more complex deep learning architectures. By establishing these baselines, we quantify the information gain provided by temporal dynamics and spatial geometric modeling.

The evaluation protocol was applied consistently across datasets: each radar frame was compressed into a fixed-size vector of 30 statistical moments. We assessed several classical classifiers, ranging from linear models to ensemble tree-based methods and shallow multi-layer perceptrons [6, 5, 3, 4, 10].

6.1 Classical ML Performance on RadHAR

For the RadHAR dataset, which contains macroscopically distinct motions, we evaluated the classical models on a strict frame-by-frame basis, explicitly removing temporal context.

Model	Accuracy (%)	Macro F1 (%)
Logistic Regression	52.1	51.3
Linear SVM	51.8	50.6
Random Forest	72.9	72.5
XGBoost	73.9	73.4
MLP baseline	74.9	74.6

Table 8: Performance of classical ML models on RadHAR, frame-by-frame.

The performance ceiling of approximately 75% confirms that statistical extraction is efficient for distinct activities, but evaluating frames in isolation leaves a significant margin of error.

Missing local file: MATRICERADHAR.PNG

Figure 6: Confusion matrix of the best classical ML classifier on RadHAR.

6.2 Classical ML Performance on DGUHA

The DGUHA dataset introduces a higher level of complexity with seven subtle classes. Ensemble tree-based models perform best, with Random Forest reaching 71.4% accuracy.

Model	Accuracy (%)	Macro F1 (%)
Logistic Regression	67.7	67.0
Linear SVM	59.6	58.0
Random Forest	71.4	69.9
XGBoost	70.8	70.0
MLP baseline	63.4	63.9

Table 9: Performance of classical ML models on DGUHA.

Confusion between geometrically similar motions highlights the limits of global statistical summaries for fine-grained gestures.

Missing local file: MATRICEDGUHA.PNG

Figure 7: Confusion matrix of the Random Forest classifier on DGUHA.

6.3 The Generalization Bottleneck on MARS

MARS reveals the critical limitations of statistical approaches. Under a random split, XGBoost reaches 61.7% accuracy, but the macro F1-score collapses to 23.8%, indicating a prediction-collapse trap.

Model	Accuracy (%)	Macro F1 (%)
Logistic Regression	40.3	15.3
Linear SVM	37.0	14.5
Random Forest	58.4	22.6
MLP baseline	37.9	14.2
XGBoost	61.7	23.8

Table 10: Performance of classical ML models on MARS with a random split.

Classical pipelines fail under subject-isolated evaluation because they overfit to physical attributes rather than geometric movement shapes. This spatial blindness motivates advanced spatial architectures such as DGCNN.

Missing local file: MATRICEMARSNOR.PNG

Figure 8: Confusion matrix on MARS with Random Forest, exposing predictive collapse.

6.4 Summary

The progression is clear: coarse macro-motions such as RadHAR can be partly solved with statistical moments, fine-grained actions such as DGUHA require non-linear ensemble methods, and symmetric cross-subject motions such as MARS demand local 3D structural modeling. These results confirm that architectural complexity is a functional requirement for robust human activity recognition, not merely an optimization detail.

7 MOMENTS

Student contribution: Meriem Mellouki and Zaineb Tarhda

7.1 Motivation

The sketching approach described in Section 3 compresses each frame into a fixed-size vector via a random Fourier feature map. A conceptually simpler alternative is to summarize the empirical distribution of points in a frame by its *statistical moments*: mean, variance, skewness, kurtosis, and range. This representation is fully deterministic, requires no learned or random parameters, and is permutation-invariant by construction.

While statistical moments cannot, in general, uniquely characterize an arbitrary distribution, a classical result in probability theory guarantees that the first $2N$ moments uniquely determine any discrete distribution supported on at most N points [1]. Since each radar frame typically contains fewer than 30 points, a moderate number of moments is therefore *sufficient to characterize the frame exactly*, up to point-set permutation. This property makes the moment representation particularly well-suited to the sparse radar setting, where voxel-based or Fourier-based approaches would introduce unnecessary complexity.

7.2 Preprocessing

Feature selection. Each detected point carries seven raw features: x, y, z (Cartesian coordinates), range r , radial velocity v , bearing θ , and intensity I . As noted in Section 1.1, Cartesian coordinates and polar features are geometrically redundant. We found it sufficient to work with a compact five-dimensional per-point representation:

$$p_i = (x_i, y_i, z_i, v_i, I_i) \in \mathbb{R}^5.$$

Moment extraction. For a frame $\mathcal{F}_t = \{p_1, \dots, p_{N_t}\}$, we compute six statistics along each of the $d = 5$ feature dimensions:

Statistic	Definition
Mean	$\mu_k = \frac{1}{N_t} \sum_i p_{ik}$
Std. dev.	$\sigma_k = \sqrt{\frac{1}{N_t} \sum_i (p_{ik} - \mu_k)^2}$
Skewness	$\gamma_{1,k} = \frac{1}{N_t} \sum_i \left(\frac{p_{ik} - \mu_k}{\sigma_k} \right)^3$
Kurtosis	$\gamma_{2,k} = \frac{1}{N_t} \sum_i \left(\frac{p_{ik} - \mu_k}{\sigma_k} \right)^4 - 3$
Minimum	$\min_i p_{ik}$
Maximum	$\max_i p_{ik}$

Concatenating these statistics across all $d = 5$ dimensions yields a fixed-size descriptor

$$s_t = [\mu_1, \sigma_1, \gamma_{1,1}, \gamma_{2,1}, \min_1, \max_1, \dots] \in \mathbb{R}^{30},$$

regardless of the number of points N_t in the frame. This descriptor serves as the input to the classifier described below.

Temporal aggregation. In contrast to the sketching pipeline, which operates over a 2-second window of 60 frames and feeds a $(60, m)$ tensor to a sequential model, the moment baseline treats each frame *independently*: a single frame is compressed into one 30-dimensional vector, which is passed directly to an MLP. This amounts to ignoring the temporal structure of the recording. The implications for performance and a discussion of how temporal information could be reintroduced are given in Section 7.4.

Standardization. The 30-dimensional feature vector is standardized to zero mean and unit variance using statistics computed on the training set only, following the same protocol as in Section 3.1.

7.3 Architecture: Multi-Layer Perceptron

Since the 30-dimensional moment vector has no spatial or sequential structure, convolutional or recurrent architectures bring no benefit here. We therefore use a fully-connected MLP with three hidden layers:

Layer	Output dim	Activation	Note
Input	30	—	Standardized moment vector
Dense 1	128	ReLU	
Dense 2	64	ReLU	
Dense 3	32	ReLU	
Output	5	Softmax	One logit per activity class

Table 11: MLP architecture used for the moment-based baseline.

The model is trained with early stopping (patience of 10 epochs, validation fraction 15%) to prevent overfitting on the relatively small dataset. No dropout or weight decay was applied, as the model is already severely regularized by the low input dimensionality.

7.4 Results and Discussion

Table 12 reports per-class precision, recall and F1-score on the RadHAR test set (58 samples, 5 classes).

Activity	Precision	Recall	F1-score	Support
Boxing	0.52	0.85	0.65	13
Jack	0.67	0.20	0.31	10
Jump	0.62	0.45	0.53	11
Squats	1.00	0.20	0.33	10
Walk	0.58	1.00	0.74	14
Overall accuracy		58.6%		

Table 12: Per-class performance of the moment-based MLP on RadHAR (test set).

The model achieves **58.6%** accuracy, well above the random baseline of 20%. Locomotor activities with a clear spatial signature (*walk*: 100% recall; *boxing*: 85% recall) are well captured by the moment representation. By contrast, activities involving repetitive vertical motion confined to a small spatial region (*squats*, *jack*) are poorly recalled (20%), likely because

a single-frame moment descriptor cannot encode the periodicity that distinguishes these activities from a static standing posture.

This limitation is a direct consequence of the *temporal blindness* of the current approach. Integrating temporal information — for instance by computing moments over a 2-second sliding window rather than per frame, or by feeding the sequence of per-frame descriptors to an LSTM — is the most natural next step and is expected to close a significant part of the gap with the sequential architectures described in Section 3.

8 RESULTS

8.1 Comparison

Table 13 summarizes the comparison grid used for the main experiments.

8.2 Discussion

RadHAR near-ceiling performance and its caveat. PointNet V1 and V2 both reach near-ceiling performance on RadHAR, with V2 averaging 96.5% accuracy and V1 averaging 95.5%. These numbers look strong but should be interpreted carefully: RadHAR was recorded by only two subjects, so a classifier can achieve high accuracy by recognizing body-shape or gait signatures that are specific to those individuals rather than by learning the activity itself. The practical implication is that RadHAR accuracy figures are systematically optimistic relative to what one would expect on an unseen subject. The two-subject limitation is a ceiling on how much RadHAR results can say about generalization.

Temporal attention provides no lift on RadHAR. V2 improves over V1 by about 1.0 percentage point in accuracy (96.49% vs. 95.48%) and 0.0094 macro F1 (0.9656 vs. 0.9562) on RadHAR. This gain is modest compared with the overall level of performance, which suggests that the temporal attention mechanism helps slightly but is not the main bottleneck. For activities like walking or boxing, a single-frame point cloud already carries a distinctive spatial signature, so modeling how the cloud evolves over time provides diminishing returns on a saturated problem.

Vanilla PointNet outperforms the temporal variant on MARS. On the harder, subject-isolated MARS benchmark, V1 (89.0%, macro F1 0.790) actually outperforms V2 (84.6%, macro F1 0.752). This reversal is counterintuitive and suggests a training-efficiency issue: V2’s additional attention parameters require more data to optimize reliably, and the MARS training set (3 subjects, 383 windows) may simply not be large enough to take advantage of temporal modeling. It also highlights that architectural complexity is not always beneficial in low-data regimes.

PointNet V2 helps on DGUHA. On DGUHA (7 classes), PointNet V2 averages 92.8% accuracy across five seeds, compared with 91.9% for PointNet V1. The temporal attention layer therefore provides a modest gain on this longer-window

dataset, where motion dynamics are more informative than in shorter binary settings.

The Moments MLP gap quantifies the value of temporal context. The moment-based MLP reaches only 58.6% on RadHAR, a gap of ~ 38 percentage points below PointNet V1. Since the MLP is temporally blind (it classifies one frame at a time), this gap directly quantifies how much information is carried by the temporal dynamics of the point cloud. Activities that are spatially ambiguous in a single frame—squats and jumping jacks are both vertical, periodic motions with a small spatial footprint—require temporal context to be disambiguable. This is the single clearest argument in the results for using sequence-aware architectures rather than frame-independent baselines.

Dataset	Architecture	Params	Epochs	Test acc.	Test macro F1
RadHAR	Linear SVM	—	—	0.5180	0.5060
	Logistic Regression	—	—	0.5210	0.5130
	Moments MLP	14,469	—	0.5860	0.5120
	Random Forest	—	—	0.7290	0.7250
	XGBoost	—	—	0.7390	0.7340
	MLP baseline	—	—	0.7490	0.7460
	SK-DGCNN	3,014,547	200	0.8877	0.8879
	Pointwise Attention Pooling	74,247	11	0.9520	0.9523
	PointNet V1	3,341,909	40	0.9548	0.9562
	PointNet V2	3,358,357	40	0.9649	0.9656
RaDataTouille	Sketching Transformer	791,000	30	0.9640	0.9601
	Pointwise Attention Pooling	74,052	11	0.7646	0.7643
	PointNet V2	3,362,294	40	0.8017	0.7998
	PointNet V1	3,345,766	40	0.8257	0.8203
	Moments MLP	—	—	—	—
DGHUA	Sketching Transformer	791,000	30	0.8422	—
	Linear SVM	—	—	0.5960	0.5800
	MLP baseline	—	—	0.6340	0.6390
	Logistic Regression	—	—	0.6770	0.6700
	XGBoost	—	—	0.7080	0.7000
	Random Forest	—	—	0.7140	0.6990
	Pointwise Attention Pooling	74,377	12	0.9161	0.9166
	PointNet V1	3,341,008	40	0.9193	0.9205
	PointNet V2	3,357,428	40	0.9279	0.9274
	SK-DGCNN	3,016,979	—	0.8238	0.8232
MARS	Moments MLP	—	—	—	—
	Sketching Transformer	—	—	—	—
	Linear SVM	—	—	0.3700	0.1450
	MLP baseline	—	—	0.3790	0.1420
	Logistic Regression	—	—	0.4030	0.1530
	Random Forest	—	—	0.5840	0.2260
	XGBoost	—	—	0.6170	0.2380
	Pointwise Attention Pooling	74,572	12	0.8251	0.8276
	PointNet V2	3,363,399	40	0.8455	0.7524
	PointNet V1	3,346,915	40	0.8900	0.7900
	Moments MLP	—	—	—	—
	Sketching Transformer	—	—	—	—

Table 13: Global comparison table for the studied datasets and architectures.

9 CONCLUSION

This study benchmarked six families of architectures—statistical moments, classical ML baselines, a sketching Transformer, PointNet (V1 and V2), a pointwise attention pooling model, and a two-stage skeleton pipeline based on SK-DGCNN—on four mmWave radar point-cloud datasets covering 2 to 10 human activity classes.

PointNet variants are the most robust choice across datasets. PointNet V1 and V2 score highest or near-highest on all four datasets. V2 (with temporal attention via RoPE)

consistently outperforms V1 when windows are long and motion dynamics are informative (RadHAR, DGUHA), but V1 is stronger in low-data regimes (MARS, RaDataTouille), where the extra attention parameters become a liability rather than an asset. The lightweight Pointwise Attention Pooling model achieves competitive results (95.2% on RadHAR, 91.6% on DGUHA) at a fraction of the parameter count, making it attractive for embedded or latency-sensitive deployments.

The skeleton pipeline underperforms despite theoretical appeal. The SK-DGCNN two-stage approach achieves 88.8% on RadHAR and 87.4% on DGUHA, trailing the best direct classifiers by 5–8 pp. The main bottleneck is error propagation: mistakes in the radar-to-skeleton regression stage cannot be corrected by the downstream classifier, which sees only the predicted skeleton. We also found that the 99.5% results claimed in the original SK-DGCNN paper result from data leakage—the published split overlaps training windows into the test set, making the evaluation substantially easier than a clean subject-isolated protocol.

Architecture complexity must match the data regime. The Sketching Transformer reaches 96.4% on RadHAR with a lightweight preprocessing pipeline, demonstrating that compact random-feature representations are a strong alternative to raw point-cloud processing when training data is ample.

Limitations and future directions. All four datasets suffer from small subject pools: RadHAR has only two subjects, MARS three, and our own RaDataTouille three. Accuracy numbers on these benchmarks measure in-distribution performance rather than true generalization, and near-ceiling scores on RadHAR should be read accordingly. Future work should prioritize collecting larger, subject-diverse recordings and evaluating under strict leave-one-subject-out protocols. Without this shift in evaluation protocol, improvements in model design will be difficult to separate from hidden dataset shortcuts. On the modeling side, integrating temporal information into the moment baseline (e.g. computing moments over a sliding window or feeding per-frame descriptors to an LSTM) and improving the skeleton regression stage with uncertainty-aware training are natural next steps that could close the remaining gaps.

A APPENDIX

A.1 Transformer architecture diagram

Missing local file:
transformer_architecture.png

Figure 9: Architecture of our Transformer encoder for RadHAR classification. The input is a sequence of 60 sketches of dimension $m = 256$, to which a fixed sinusoidal positional encoding is added. The sequence is processed by $L = 3$ identical encoder blocks, each with $h = 4$ attention heads and a feed-forward expansion ratio of 2, temporally pooled, and classified into the 5 activity classes.

A.2 PointNet V1 architecture and hyperparameters

PointNet V1 is the vanilla direct point-cloud classifier used in `src/pointnetv1/`. For RadHAR, one training sample has shape $(60, 32, 4)$: 60 frames, 32 points per frame, and the four features x, y, z , velocity. The input windows use a stride of 60 frames. Features are standardized with training-set statistics and clipped at ± 4 . Training uses batch size 16, dropout 0.5, Adam with base learning rate 10^{-3} , momentum parameter 0.9, five warmup epochs, label smoothing 0.2, T-Net orthogonality regularization coefficient 10^{-3} , and gradient accumulation 1.

Module	Layers	Output
T-Net block 1	$4 \rightarrow 64 \rightarrow 128 \rightarrow 1024 \rightarrow 512 \rightarrow 256 \rightarrow 4^2$	4×4 transform
Encoder block 1	shared MLP $4 \rightarrow 64 \rightarrow 64$	$(60, 32, 64)$
T-Net block 2	$64 \rightarrow 64 \rightarrow 128 \rightarrow 1024 \rightarrow 512 \rightarrow 256 \rightarrow 64^2$	64×64 transform
Encoder block 2	shared MLP $64 \rightarrow 64 \rightarrow 128 \rightarrow 1024$	$(60, 32, 1024)$
Pooling	max over points, mean over frames	1024
Classifier	$1024 \rightarrow 512 \rightarrow 5$ with dropout	5 logits

Table 14: PointNet V1 module layout.

A.3 PointNet V2 architecture and hyperparameters

PointNet V2 uses the same data shape, preprocessing, classifier head and training hyperparameters as V1, but adds temporal attention inside each encoder block before the shared MLP. The attention is applied to each point trajectory across the 60-frame window. With four input features the implementation uses two heads in the first block; with 64 features it uses eight heads in the second block. Queries and keys receive rotary positional embeddings so that attention is aware of relative frame position.

Module	Layers	Output
Attention block 1	T-Net $4 \rightarrow 4^2$, MHA 2 heads, MLP $4 \rightarrow 64 \rightarrow 64$	$(60, 32, 64)$
Attention block 2	T-Net $64 \rightarrow 64^2$, MHA 8 heads, MLP $64 \rightarrow 64 \rightarrow 128 \rightarrow 1024$	$(60, 32, 1024)$
Pooling	max over points, mean over frames	1024
Classifier	$1024 \rightarrow 512 \rightarrow 5$ with dropout 0.5	5 logits

Table 15: PointNet V2 module layout.

A.4 SK-DGCNN architecture and hyperparameters

The SK-DGCNN skeleton regressor is implemented in `src/SK-dgcnn/MARS_skdgcnn.ipynb`. Each sample is one MARS radar frame with shape $(25, 5)$, using x, y, z , velocity, intensity. The model predicts 19 centered

skeleton joints with two coordinates per joint, therefore 38 scalar outputs. Training uses batch size 16, test batch size 64, 150 epochs, Adam with learning rate 10^{-3} and momentum parameter 0.5, multi-step decay at epochs 15 and 20 with factor 0.1, dropout 0.4, $k = 20$ neighbors, embedding dimension 512, ECA kernel size 3, and a validation fraction of 0.2. The loss is a sequential multitask combination of MSE and KL-divergence over the z then x skeleton axes, changing phase every 75 epochs.

Module	Layers	Output
EdgeConv 1	graph feature 10, Conv2d 10 $\rightarrow$ 64, BN, LeakyReLU, ECA	(64, 25)
EdgeConv 2	graph feature 128, Conv2d 128 $\rightarrow$ 64, BN, LeakyReLU, ECA	(64, 25)
EdgeConv 3	graph feature 128, Conv2d 128 $\rightarrow$ 128, BN, LeakyReLU, ECA	(128, 25)
Embedding	concat 256, Conv1d 256 $\rightarrow$ 512, BN, LeakyReLU, ECA	(512, 25)
Pooling	adaptive max pool + adaptive avg pool	1024
Joint trunk	Linear 1024 $\rightarrow$ 512 $\rightarrow$ 19 $\cdot$ 128 with dropout	(19, 128)
Heads	Linear 128 $\rightarrow$ 1 for x , Linear 128 $\rightarrow$ 1 for z	38 values

Table 16: SK-DGCNN skeleton regressor layout.

REFERENCES

- [1] N. I. Akhiezer. *The Classical Moment Problem and Some Related Questions in Analysis*. Oliver and Boyd, Edinburgh and London, 1965.
- [2] Sizhe An and Umit Y. Ogras. Mars: mmwave-based assistive rehabilitation system for smart healthcare. *ACM Transactions on Embedded Computing Systems*, 20(5s):72:1–72:22, 2021.
- [3] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [5] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [6] D. R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2):215–232, 1958.
- [7] Gawon Lee and Jihie Kim. Mtgea: A multimodal two-stream gnn framework for efficient point cloud and skeleton data alignment. *Sensors*, 23(5):2787, 2023.
- [8] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 77–85, 2017.
- [9] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, volume 20, pages 1177–1184, 2007.
- [10] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [11] Akash Deep Singh, Sandeep Singh Sandha, Luis Garcia, and Mani Srivastava. Radhar: Human activity recognition from point clouds generated through a millimeter-wave radar. In *Proceedings of the 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems*, pages 51–56, New York, NY, USA, October 2019. Association for Computing Machinery. mmNets ’19, co-located with MobiCom ’19: The 25th Annual International Conference on Mobile Computing and Networking.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [13] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics*, 38(5):1–12, 2019.
- [14] Zihan Zhang, Aman Anand, and Farhana Zulkernine. Sk-dgcnn: Human activity recognition from point cloud data with skeleton transformation. *Machine Learning with Applications*, 23:100847, 2026.